

경사하강법(Gradient Descent)

정의

가중치 갱신 과정

편미분과 연쇄 법칙(Chain Rule)

옵티마이저(Optimizer)

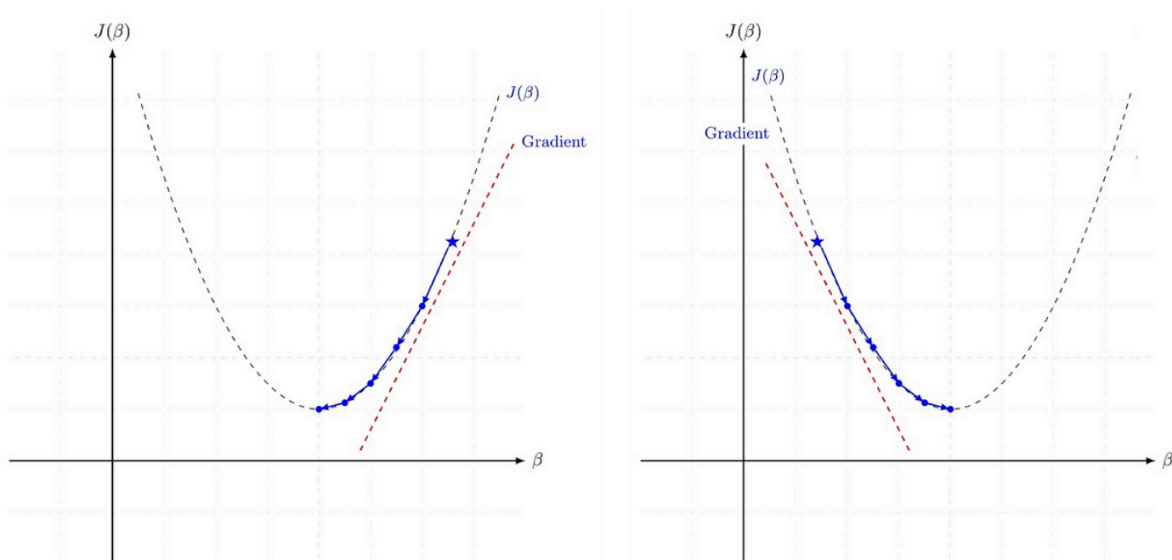
확률적 경사 하강법(Stochastic Gradient Descent, SGD)

미니 배치(Mini-batch GD) 경사 하강법

그 외 옵티마이저들

정의

- 역전파는 손실 함수를 통해서 얻은 손실 정보를 출력층부터 입력층까지 전달하여 매 단계 가중치가 어떻게 조정되어야 할지 구하는 과정
- 가중치를 조정시 사용되는 방법이 바로 **경사 하강법(GD; Gradient Descent)**이다.



손실함수, 가로축 가중치, 세로축 손실함수의 값

- 손실 함수의 값을 줄이기 위해서는 맨 아래에 위치한 최소값을 향해서 가야함
- 손실 함수의 기울기가 작아지는 방향으로 가중치를 갱신하는 것
- 기울기는 손실 함수를 미분한 함수, 즉 손실 함수의 도함수가 바로 기울기이며 손실 함수의 도함수를 계산하여 이 값이 작아지도록 가중치를 조정

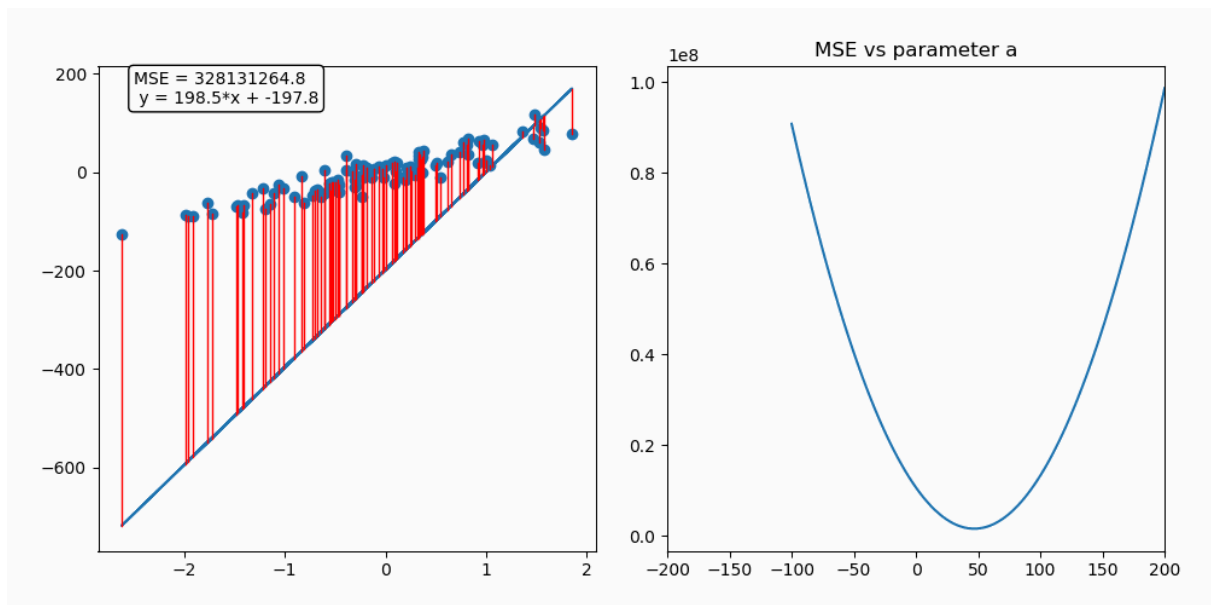
가중치 갱신 과정

기울기 반대 방향으로 η 배 만큼 이동

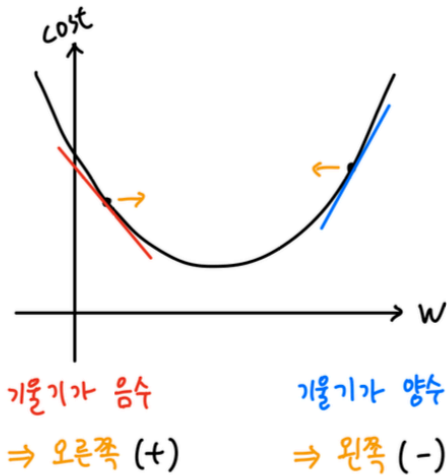
$$\theta_{i,j+1} = \theta_{i,j} - \eta \frac{\partial}{\partial \theta_{i,j}} J(\theta)$$

새롭게 갱신된 가중치 갱신 전 가중치 학습률 해당 지점에서의 기울기

- i 번째 가중치인 θ_i 가 갱신되는 모습을 수식으로 표현한 것
- η 는 경사를 얼마나 큰 보폭으로 내려갈지 결정
- $\theta_{i,j}$ 의 j 는 iteration을 의미
- $J(\theta)$ 는 가중치 θ 에 대한 손실 함수
- ∂ 는 편미분 기호



- 기울기가 작아지는 방향으로 가중치를 움직여야 하므로 기울기 값에 음의 부호(-)를 붙임.
- 양수 기울기는 음수로, 음수 기울기는 양수가 되어 매 갱신마다 가중치가 손실 함수의 최소값에 가까워지도록 만듦



$$W_{t+1} \leftarrow W_t - \eta \frac{\partial L}{\partial W}$$

편미분과 연쇄 법칙(Chain Rule)

- 각각의 가중치를 갱신하기 위한 방법으로 편미분과 Chain rule (연쇄 법칙)을 사용
- 특정 가중치 (θ_i) 에 대한 기울기는 아래 식과 같이 손실 함수를 해당 가중치로 편미분하여 구할 수 있음

$$\frac{\partial}{\partial \theta_i} J(\theta)$$

- 모든 가중치에 대한 값을 구하기 위해 바로 **Chain rule**이 적용됨. 연쇄 법칙이란 아래 식과 같이 특정 변수에 대한 (편)미분 값을 다른 변수의 미분을 사용하여 나타낼 수 있는 방법

옵티마이저(Optimizer)

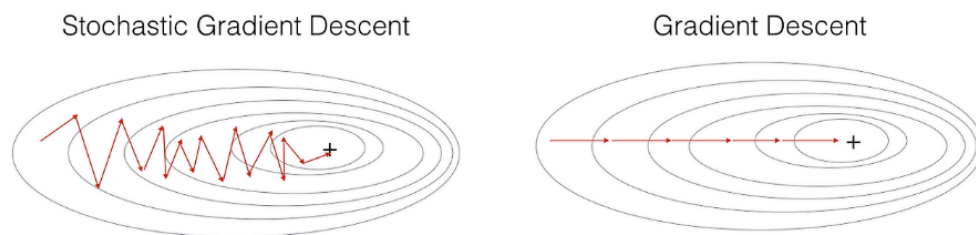
$$\frac{\partial J(\theta)}{\partial \theta_i} = \frac{\partial J(\theta)}{\partial \theta_x} \cdot \frac{\partial \theta_x}{\partial \theta_i} = \frac{\partial J(\theta)}{\partial \theta_x} \cdot \frac{\partial \theta_x}{\partial \theta_y} \cdot \frac{\partial \theta_y}{\partial \theta_i}$$

- 경사를 내려가는 방법을 최적화하는

- 일반적인 경사 하강법(GD)에서는 모든 입력 데이터에 대한 손실 함수의 기울기를 계산한 후에 가중치를 업데이트 함. 이때 Iteration 마다 모든 데이터를 다 사용하는데 입력 데이터가 적다면 이 방법으로도 빠르게 가중치를 금방 갱신함
- 하지만 실제로는 이보다 훨씬 더 큰 데이터를 다루기 때문에 손실을 계산하는 과정이 굉장히 오래 걸림
- 이를 보완하기 위해 **확률적 경사 하강법**과 **미니 배치(Mini-batch) 경사 하강법**

확률적 경사 하강법(Stochastic Gradient Descent, SGD)

- 확률적 경사 하강법은 전체 데이터에서 하나의 데이터만 뽑아서 손실 계산 및 역전파 과정을 수행하는 방식
- 한 번의 iteration마다 1개의 데이터만 사용
- 특징
 - 데이터를 1개만 쓰다 보니 가중치를 업데이트하는 속도는 매우 빠름
 - 1개만 쓰기 때문에 학습 과정에서 경사 하강이 불안정함



SGD와 GD 비교

미니 배치(Mini-batch GD) 경사 하강법

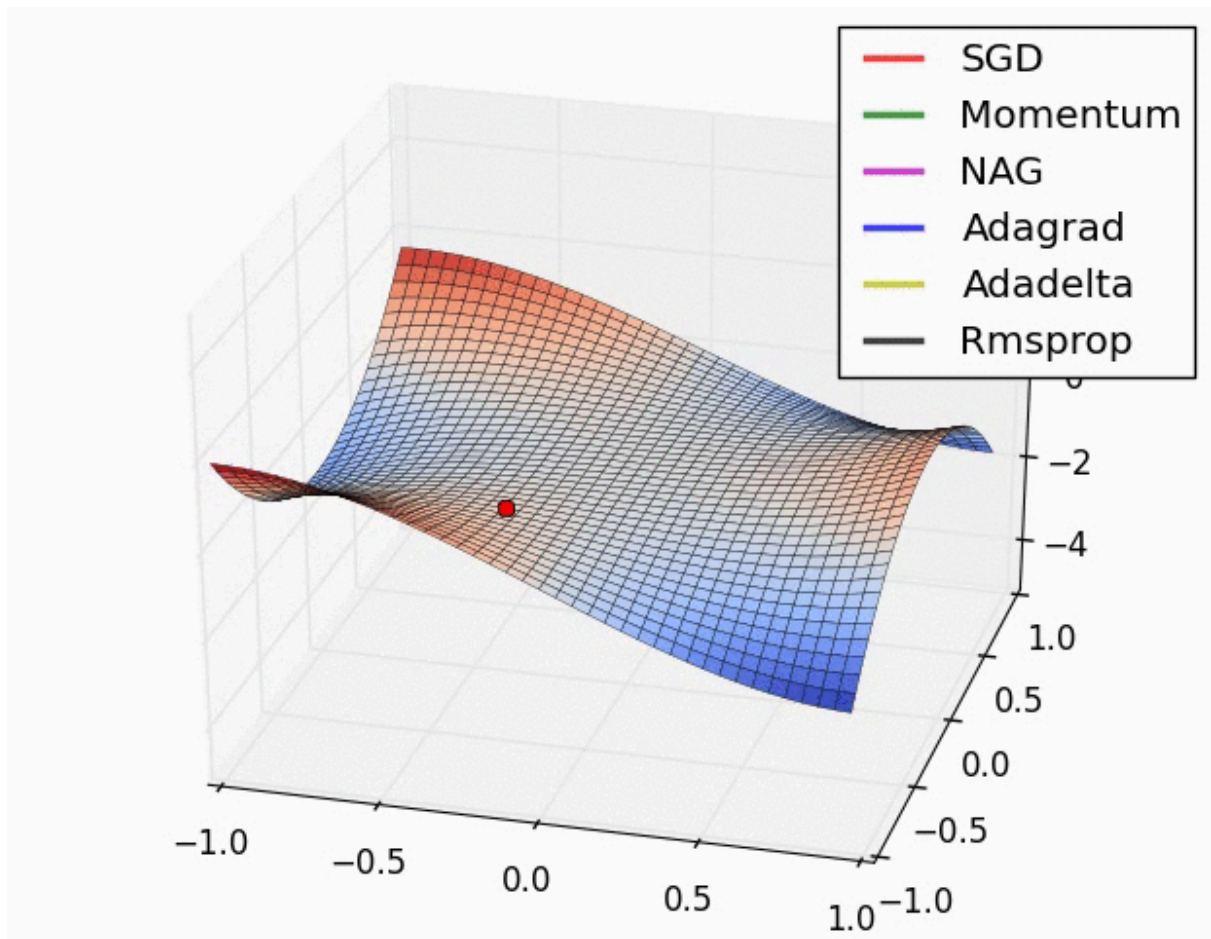
- 전체 데이터를 작게 쪼개어서 학습 과정에 보내는 방식
- 어느 정도의 크기로 나누는가에 따라 학습 속도나 가중치 갱신의 최적점 도달 등의 결과가 달라짐
- 여기서 데이터를 나누어주는 크기를 **배치 사이즈(Batch size)** 라고 한다.
- 배치 사이즈(Batch size)
 - 미니 배치 경사 하강법에서 사용하는 미니 배치의 크기
 - 일반적으로 2의 배수로 설정
 - 메모리 용량에 여유가 있다면 가능한 한 큰 배치 사이즈를 쓰는 것이 안정적인 학습 진행에 유리함

- 배치 사이즈가 작을 때
 - 이로 인해 최적점 도달에 많은 iteration이 필요함 동시에 불안정
 - 노이즈(noise)가 많기 때문에 지역 최적점(Local Minima)에서 빠져나와 전역 최적점(Global Minima)에 도달할 확률이 높아진다는 장점
 - 배치 사이즈가 작아질수록 경사 하강법의 가중치 갱신이 불안정해짐
- 배치 사이즈가 클 때
 - 배치 사이즈가 커질수록 가중치 갱신이 안정적
 - 최적점 도달에 비교적 적은 iteration만 필요
 - 배치 사이즈를 너무 크게 설정하면 메모리 용량을 초과해버리는 Out-of-Memory 문제가 발생할 수도 있으므로 적당한 배치 사이즈 설정 필요
- 데이터 수와 배치 사이즈, iteration의 관계

$$\# \text{ of Data} = \text{Batch size} \times \text{Iteration}$$

그 외 옵티마이저들

GD와 SGD, Mini-batch GD 이후로 아래와 같이 다양한 옵티마이저들이 나옴



- 옵티마이저별 특징
 - Momentum : 가던 방향으로 더 빨리 잘 가게 밀어주기
 - AdaGrad : 모르는 곳은 확확 둘러보고, 아는 곳은 자세하게 들여다 보기
 - RMSProp : 매 발걸음마다 내가 가는 이 길이 맞는지 아닌지 점검하며 가기
 - AdaDelta : AdaGrad의 개선 버전. 자세히 보는건 좋은데 멈추진 말고 계속 가면서 보기
 - Adam : AdaGrad와 RMSProp의 퓨-전(fusion). AdaGrad처럼 상황에 따라 보폭 조절하고, RMSProp처럼 제대로 가고 있는지 봐가면서 움직이기
- Adam이 현재 가장 널리 사용되고 있지만 데이터의 성질과 해결할 문제 등을 고려하여 옵티마이저를 선택해야함